TEMA 2

NET FRAMEWORK

Desarrollo Interfaces



IES HENRI MATISSE

IES HENRI MATISSE

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

TEMA 2 – PLATAFORMA .NET

Contenido

1. NET 4
1.1. Características .NET 4
1.2. Componentes 5
2. WPF 7
2.1. XAML 9
2.2. Codificado con C# 10
3. Eventos comunes 11
3.1. Teclado 11
3.2. Raton 11
3.3. Window 12
3.4. Foco 12
3.5. Item 12
3.6. Drag and Drop 12
4. Manejador de eventos 12
5. Layout 14
5.1. GridPanel 15
5.2. StackPanel 17
5.3. DockPanel 18
5.4. WrapPanel 19
5.5. CanvasPanel 19
6. Estilos 20
6.1. Niveles 21
6.2. Destino 23
6.3. Triggers 24
6.3.1. Property Trigger 24
6.3.2. Data Trigger 24
6.3.3. Event Trigger 25
6.3.4. MultiTrigger 25

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

7. Bibliografia 26

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

1. NET

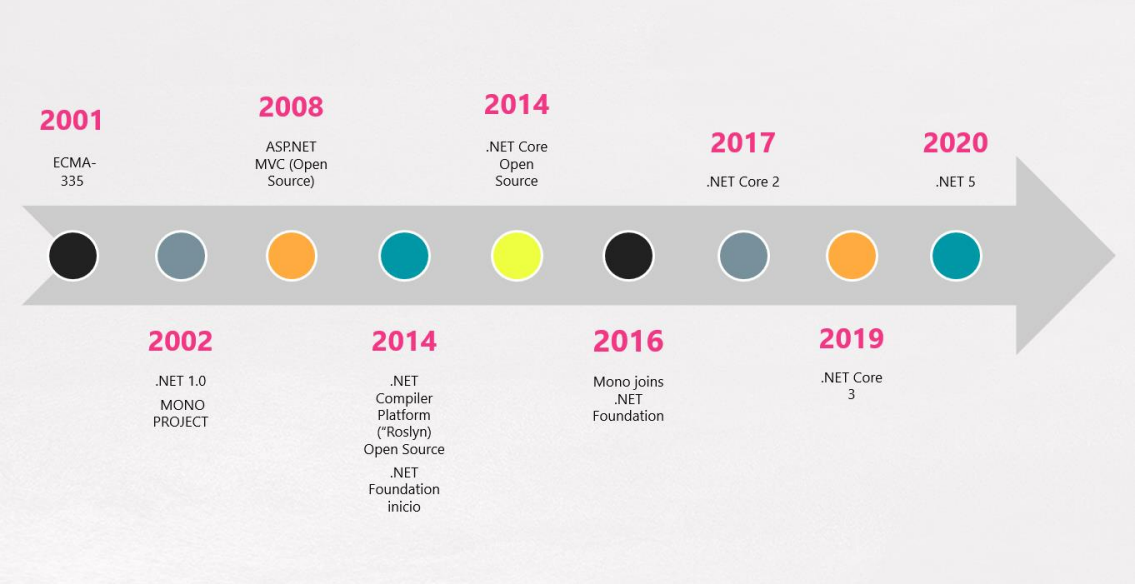
Como respuesta a la plataforma de desarrollo de Java, Microsoft construyo la suya propia que denomino **.NET**.

NET Framework es un entorno de ejecución administrado que proporciona diversos servicios a las aplicaciones en ejecución. Se trata de un entorno de desarrollo de aplicaciones que nos aporta herramientas, bibliotecas y entornos de ejecución para nuestras aplicaciones desarrolladas en distintos lenguajes de programación.

Las aplicaciones desarrolladas con esta plataforma son interpretadas por un entorno común de ejecución, similar a la máquina virtual de Java.

NET Framework es un entorno de Microsoft que solamente está disponible en **Windows**, pero ante el auge del software libre, Microsoft decidió reescribir este entorno para que también pudiera ser utilizado en Linux y entornos Apple. Por lo tanto, **.NET** se trata de un entorno de **código abierto y multiplataforma** con el apoyo de una empresa como Microsoft.

A este entrono, Microsoft lo llamó .NET Core, sin embargo, ha decidido resumirlo y en la actualidad recibe el nombre de **.NET** simplemente. En la siguiente imagen tenemos la evolución de la plataforma.



1.1. Características .NET

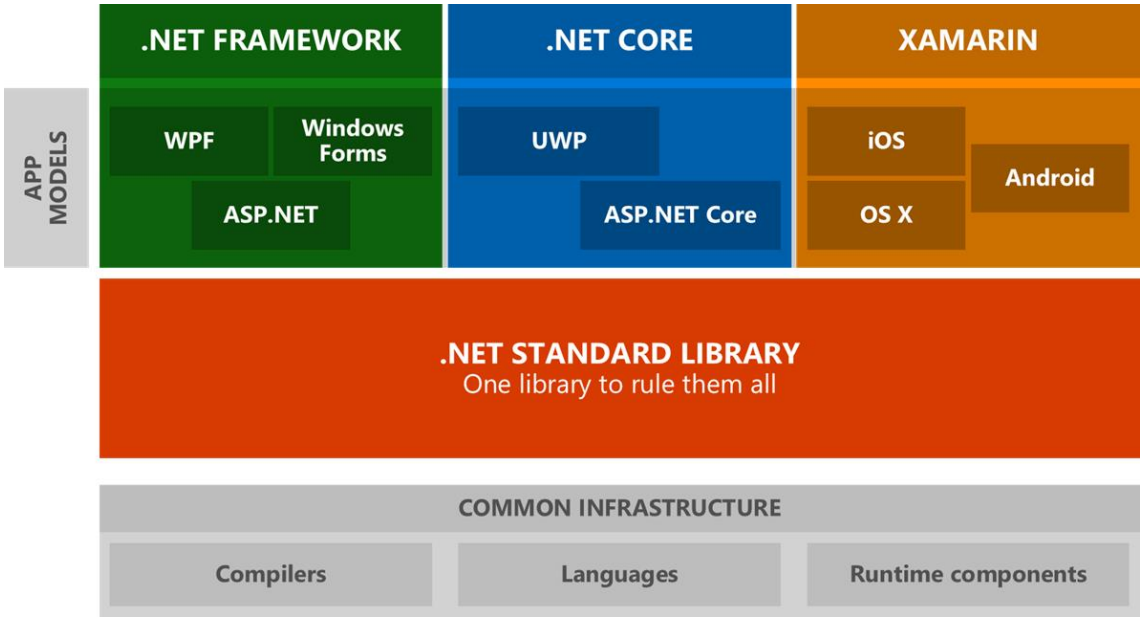
- Las aplicaciones .NET se pueden desarrollar en cualquier lenguaje de programación que se ajuste a las especificaciones.
- Soporta una extensa colección de librerías de clases independientes del lenguaje de programación para la realización de tareas comunes de acceso a datos, sobre seguridad, …
- Soporta la creación de componentes.

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

- Ofrece integración multi-lenguaje, reutilización de componentes, y herencia entre componentes desarrollados en diferentes lenguajes.
- Los tipos de objetos son especificados por el framework y no por el lenguaje.
- Soporte para multiplataforma puesto que está disponible para varios sistemas operativos.
- Permite realizar código asíncrono de forma sencilla.
- Código abierto, administrado por .NET Foundation disponible en GitHub, lo que fomenta la existencia de una amplia comunidad activa.
- Distribución modular a través de paquetes NuGet.
- Orientado al desarrollo de microservicios.

1.2. Componentes

En la siguiente figura se muestra la arquitectura y principales partes del entorno .NET



En la parte de arriba tenemos los distintos modelos para realizar aplicaciones en el entorno .NET:

- **Aplicaciones de escritorio**: en .NET Framework tenemos la posibilidad de hacerlas con la tecnología más antigua como Windows Forms o con WPF (Windows Presentation Foundation). También se puede utilizar UWP para realizar aplicaciones en la tienda de Windows. Mientras que en la plataforma .NET se puede utilizar UWP y WPF pero no hay soporte para Windows Forms.
- **Aplicaciones Web**: en este caso disponemos de ASP .NET para .NET Framework y ASP .NET Core para .NET. Esta última plataforma también nos permite Blazor que nos permite desarrollar aplicaciones web clientes sin escribir una línea en JavaScript.

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

- **Compiladores cruzados**: se trata de herramientas que se programan en un lenguaje, pero que luego el compilador lo compila para varias plataformas, como por ejemplo Xamarin que nos permite escribir una aplicación en C# y compilarla para plataformas Android, IOS o OS X. En la actualidad ha evolucionado a NET MAUI que además nos permite compilar las aplicaciones a Windows.

Luego tenemos el lenguaje intermedio que tiene la plataforma, al igual que sucede con Java, en concreto con los bytecodes. De la misma forma que en Java se utiliza la técnica de JIT (Just In Time). A este lenguaje intermedio se le denomina CLS (Common Language Specificaction).

A continuación, tenemos las bibliotecas con funcionalidades comunes para la mayor parte de aplicaciones. Al haber tantas especificaciones resultaba problemático tener varias de esas librerías según la versión que tuviéramos. Para evitar este problema se ha lanzado una nueva especificación que ponga orden. A esta especificación se le llama **.NET Standard Library**.

Se trata de una colección de código OO (orientado a objetos) que puede ser empleado desde cualquier lenguaje .NET. Define los tipos básicos, clases para la entrada/salida, seguridad, etc.

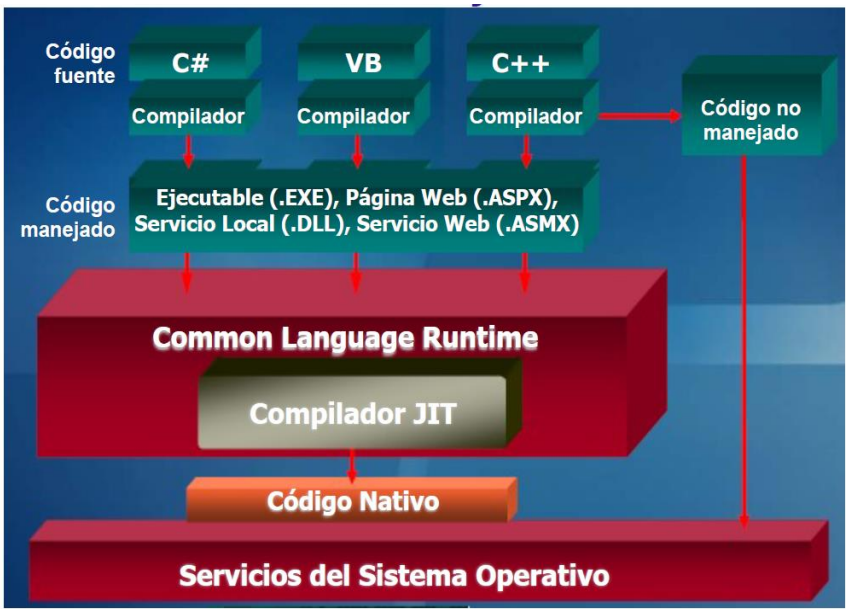
Al tener definidos los tipos de datos para todos los lenguajes, facilita el intercambio de información entre aplicaciones desarrolladas en distintos lenguajes.

Por último, tenemos la capa de CLR (Common Language Runtime) que se encarga de ejecutar las aplicaciones, de la misma forma que la máquina virtual de Java.

Un Runtime es un entorno en el que se ejecutan los programas, en concreto este se encarga de ejecutar las aplicaciones de .NET

El CLR realiza una compilación Just in Time (JIT) que traduce el código gestionado en código nativo sobre la arquitectura de hardware sobre la que se ejecuta. Es decir que interpreta el código intermedio de igual manera que hace la máquina virtual de Java.

El modelo de ejecución que realiza es el siguiente:



C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

Se encarga de tareas como cargar las clases, **recolector de basura**, control de la pila, interactuar con el sistema operativo, control de excepciones, control de seguridad o compilador de código nativo.

2. WPF

Windows Presentation Foundation (WPF) es una tecnología de Microsoft. Permite el desarrollo de interfaces de interacción en Windows.

Esta tecnología viene a sustituir a *Windows Forms* que se utilizaba en la versión 2 de .NET Framework.

WPF ofrece una amplia variedad de controles y componentes gráficos con la que es posible desarrollar aplicaciones visualmente atractivas, con facilidades de interacción que incluyen animación, vídeo, audio, documentos, navegación o gráficos 3D.

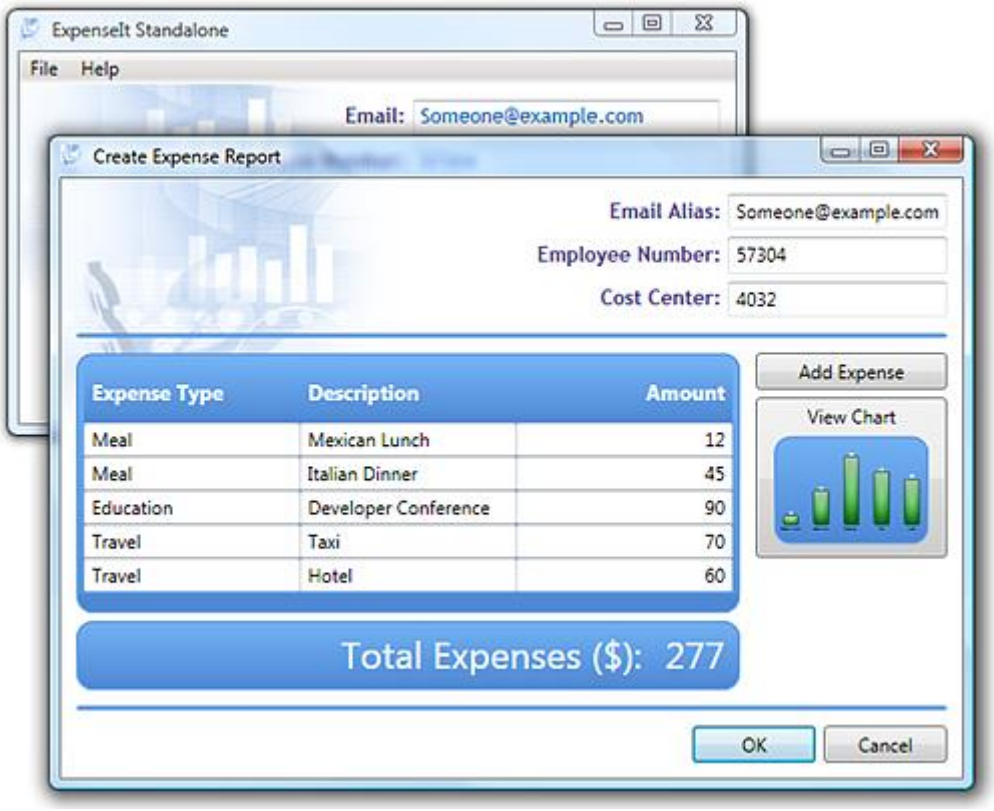
Separa claramente la definición de la pantalla o interfaz de la programación de sus funcionalidades, siguiendo de manera clara el patrón de diseño Modelo Vista Controlador.

Para especificar la interfaz utiliza un lenguaje basado en XML llamado XAML, mientras que las funcionalidades de los distintos componentes las realiza en el lenguaje elegido de entre la amplia gama que se puede utilizar.

También nos permite desarrollar dos modalidades de aplicaciones:

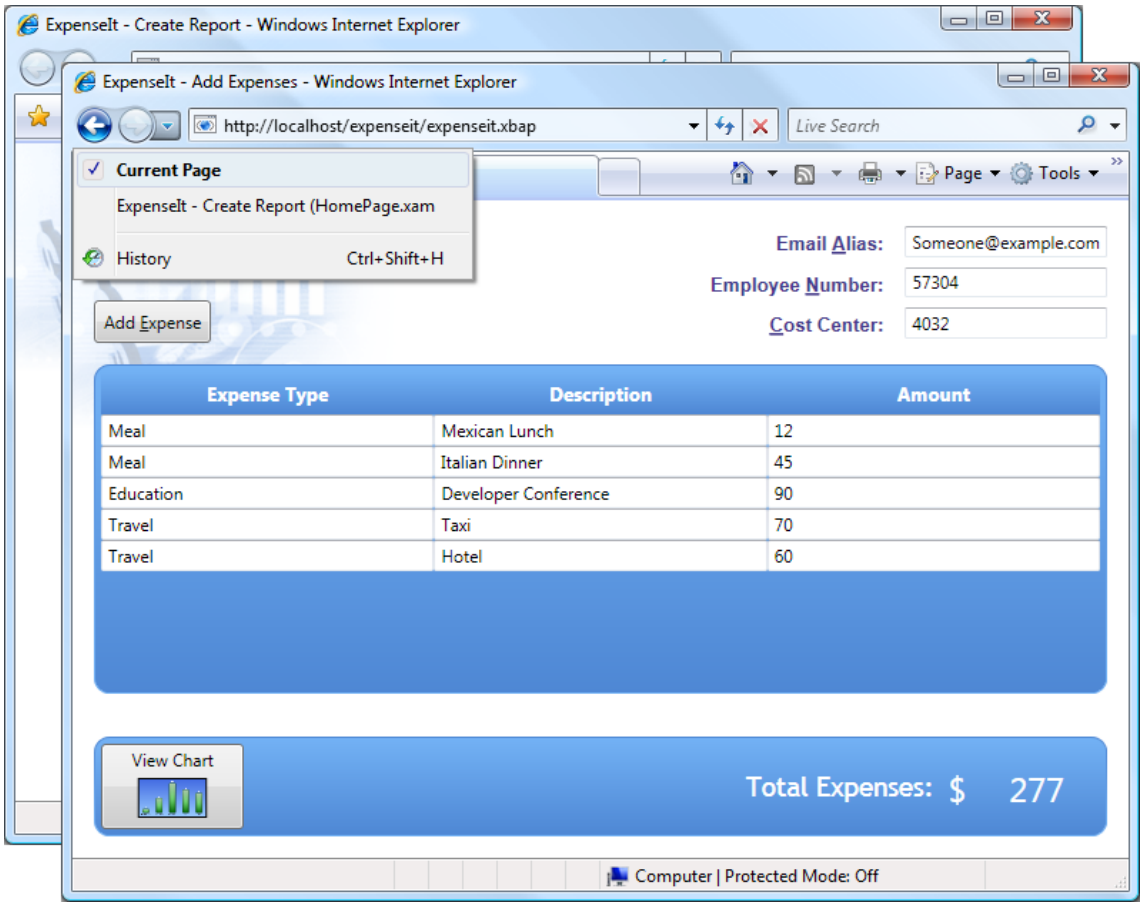
- **Aplicaciones independientes**, al estilo tradicional de Windows Forms que se instalan en el equipo cliente y se ejecutan desde él. Modalidad que he utilizado para este proyecto.

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es



- **Aplicaciones denominadas XAML Browser Applications (XBAPs)**. Son aplicaciones compuestas de páginas de navegación que se compilan y se alojan en exploradores web como Internet Explorer o Mozilla.

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es



El uso de WPF está enfocado a aplicaciones que hagan uso desde contenido multimedia (videos, audio, texto enriquecido) como para facilitar la creación de aplicaciones empresariales de "tipo escritorio" o aplicaciones web enriquecidas.

2.1. XAML

Extensible Application Markup Language es un lenguaje de marcas basado en XML que permite el diseño de una interfaz de usuario de una aplicación. XAML simplifica la creación de interfaces de usuario mediante la separación de los elementos de desarrollo y diseño de la aplicación. XAML está en archivos de texto que están separados del código de la aplicación.

XAML es fácil de leer y entender, y puede ser editado con cualquier editor de texto. Se guarda en archivos de texto con una extensión de archivo .xaml.

Los objetos declarados mediante el lenguaje XAML tienen su correspondiente instanciación en las clases del lenguaje utilizado, de manera que podemos utilizarlos en nuestro programa de la misma manera en la lo haríamos si lo definiéramos por código.

Existe una clase que se encarga de leer los objetos declarados en XAML y los crea en el entorno de ejecución del lenguaje utilizado, en nuestro caso C#. Luego desde C# podemos utilizar esos objetos para modificar su comportamiento, realizar operaciones sobre la base de datos o mostrar información.

El método que se utiliza para cargar un fichero XAML desde un programa escrito en C#, u otro lenguaje cualquiera, es ***InitializeComponent()***. Este método lee el fichero xaml, realiza un

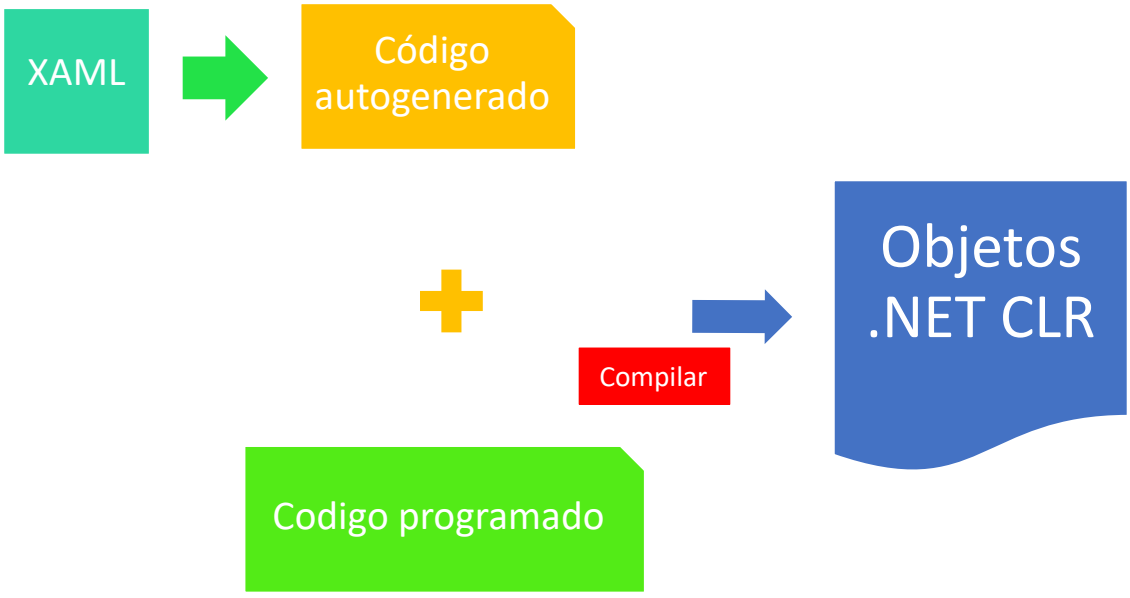
C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

parse de xml para que la sintaxis sea correcta y carga en el entorno de ejecución de la aplicación los objetos que representan a los distintos controles que se encuentran especificados en nuestra interfaz.

Cuando generamos una interfaz con XAML, el entorno crea automáticamente unas clases con el código que representa nuestros componentes o controles.

Luego unimos este código autogenerado al código que nosotros hemos realizado para formar los objetos que los identifican en nuestro CLR (máquina virtual) y poder ejecutar el programa.

En el siguiente esquema podemos observar el proceso:



A continuación tenemos un ejemplo de declaración de un par de controles en XAML.

```
<StackPanel>
    <TextBlock Margin="20">Welcome to the World of XAML</TextBlock>
    <Button Margin="10" HorizontalAlignment="Right">OK</Button>
</StackPanel>
```

Este código xml utiliza un layout o manera de colocar elementos mediante las etiquetas *StackPanel*. Luego añadimos un título mediante las etiquetas *TextBlock* con un margen de 20 px. Posteriormente añadimos un botón que alineamos a la derecha con una distancia de 10 px al margen, el texto que muestra el botón **OK**.

2.2. Codificado con C#

3. También podemos realizar lo mismo mediante C#, en el siguiente cuadro se muestra el ejemplo:

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

```
// Create the StackPanel
StackPanel stackPanel = new StackPanel();
this.Content = stackPanel;

// Create the TextBlock
TextBlock textBlock = new TextBlock();
textBlock.Margin = new Thickness(10);
textBlock.Text = "Welcome to the World of XAML";
stackPanel.Children.Add(textBlock);

// Create the Button
Button button = new Button();
button.Margin= new Thickness(20);
button.Content = "OK";
stackPanel.Children.Add(button);
```

3. Eventos comunes

Cuando se construye una aplicación con interfaz gráfica de usuario la forma de programar varía con respecto a una aplicación de consola. En una aplicación con interfaz gráfica se asocia código a diferentes eventos que puede producir el usuario cuando interactúa con la aplicación.

De manera que, cuando se genera un evento (por ejemplo, pulsar un botón, pulsar una tecla del teclado, pasar el ratón sobre una imagen, etcétera), se ejecuta el código asociado y, habitualmente, se modifica la interfaz gráfica de usuario para mostrar el resultado de esa ejecución. Cuando finaliza la ejecución de ese código asociado, la aplicación espera nuevos eventos por parte del usuario.

Dependiendo de la plataforma utilizada podemos tener distintos eventos, sin embargo, los principales suelen ser los mismos. Los principales eventos se agrupan en diferentes categorías, las cuales se describen a continuación:

3.1. Teclado

- **Key Pressed**: se activa al presionar una tecla
- **Key Release**: se activa al soltar una tecla
- **Key Typed**: se activa al teclear una tecla, es decir, primero se presiona y luego se suelta
- **Text Changed**: se activa al cambiar el texto

3.2. Raton

- **Mouse Clicked**: se activa al clickar el botón del ratón
- **Mouse Dragged**: se activa al arrastrar el ratón
- **Mouse Entered**: se activa al entrar el cursor del ratón dentro del área que forma el control
- **Mouse Exited**: se activa al salir el cursor del ratón dentro del área que forma el control
- **Mouse Moved**: se activa al mover el ratón
- **Mouse Pressed**: se activa al pulsar el botón del ratón
- **Mouse Released**: se activa al soltar el botón del ratón

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

- **Scroll Started**: se activa al iniciar el scroll, bien sea al desplazar la rueda del ratón o mediante la detección del movimiento del dedo
- **Scroll Finished**: es el evento complementario al anterior e indica la finalización del movimiento de scroll.

3.3. Window

- **WindowShown**: al hacerse visible el componente se produce el evento
- **WindowHidden**: se activa al desaparecer el componente
- **WindowMoved**: se activa al cambiar de posición el componente
- **WindowResized**: invocado cuando el componente cambia de tamaño

3.4. Foco

- **Focus**: el componente obtiene el foco
- **Blur**: el componente pierde el foco

3.5. Item

- **ItemChanged**: se activa al cambiar el elemento. Se suele utilizar en los componentes List y ComboBox para averiguar si se selecciona un ítem o elemento de la lista.

3.6. Drag and Drop

- **On Drag Detected**: se activa al detectar el movimiento de drag and drop
- **On Drag Done**: se active al finalizar la acción de arrastrar y soltar
- **On Drag Entered**: se active al iniciar el movimiento
- **On Drag Exited**: se active al finalizar el movimiento

- Existen más eventos que iremos analizando posteriormente porque ya no son comunes si no específicos de algunos componentes.

- *En el siguiente sitio web https://msdn.microsoft.com/es-es/library/cc295158.aspx tenemos una descripción breve de los eventos que se generan en WPF.*

---

4. Manejador de eventos

Tras producirse algún evento de los anteriores, nuestro programa debe de realizar alguna acción. Los eventos se notifican y se nos indica el tipo de evento y el componente sobre el que se produce.

Para **asociar el evento que se produce con nuestro código se crean unos métodos especiales llamados manejadores de eventos (event handlers)**. Estos métodos son llamados al producirse un evento y se ejecuta el código que hay en él. Si queremos gestionar algún evento lo debemos especificar en nuestro código. Vamos a ver un pequeño ejemplo y lo analizamos paso a paso.

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

En primer lugar definimos una interfaz sencilla en la que añadimos un botón:

```
<Window x:Class="WpfApplication1.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:local="clr-namespace:WpfApplication1"
        mc:Ignorable="d"
        Title="MainWindow" Height="525" Width="707.836">
    <Grid Name="pnlMainGrid" MouseUp="pnlMainGrid_MouseUp"
Background="LightBlue">
        <Button x:Name="button" Content="Button" HorizontalAlignment="Left"
Margin="247,227,0,0" VerticalAlignment="Top" Width="109" Height="47"/>

    </Grid>
</Window>
```

De momento solamente hemos creado un evento en el panel que contiene a nuestro botón, concretamente **MouseUp**. Le hemos asignado el valor ***pnlMainGrid_MouseUp*** que no es más que el nombre del método que definimos en la clase asociada a la página principal para que maneje el evento cuando se produzca. El código asociado a la ventana principal es:

```
namespace WpfApplication1
{
    /// <summary>
    /// Interaction logic for MainWindow.xaml
    /// </summary>
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void pnlMainGrid_MouseUp(object sender, MouseButtonEventArgs e)
        {
            MessageBox.Show("You clicked me at " + e.GetPosition(this).ToString());
        }
    }
}
```

Se asocia el componente y el evento que se produce sobre el componente con el método que manejará el evento. Cuando el usuario suelte el botón del ratón sobre el panel, entonces se ejecutará el método asociado, que en este caso sacará un mensaje que indicará la posición sobre la que se ha soltado el botón del ratón.

Este evento lo hemos definido manualmente en el código XAML y el código de C#, sin embargo podemos hacer que el entorno lo haga de forma automática al pulsar dos veces sobre el componente en la pantalla de diseño.

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

Esta manera de actuar nos condiciona a que el evento que se utilizará será el que el componente tiene asignado por defecto. Por ejemplo, en nuestro caso si apretamos dos veces el botón en la pantalla de diseño se asociará al evento **Click** que se activará al pinchar con el ratón el botón.

Ahora el entorno ha añadido `Click="button_Click"` en la declaración de la etiqueta asociada al botón y de paso ha generado un nuevo método en la clase controladora de la pantalla principal.

```
private void button_Click(object sender, RoutedEventArgs e)
        {
            ((Control)sender).Background = Brushes.BlueViolet;

        }
```

5. Layout

Las aplicaciones tienen una premisa clara, se pueden ejecutar en cualquier tipo de plataforma, esto que supone una gran ventaja, nos obliga a tener cuidado con el diseño de las interfaces gráficas porque los componentes se descuadrarán al cambiar de plataforma.

Para solucionarlo se introdujo el término de layout, **con esto se pretende posicionar los elementos o componentes en nuestro formulario independientemente de la fuente, de la resolución de la pantalla o las diferencias entre plataformas**.

También nos permitirá facilitar la traducción, puesto que si aumenta el tamaño de la cadena también lo hará el componente proporcionalmente.

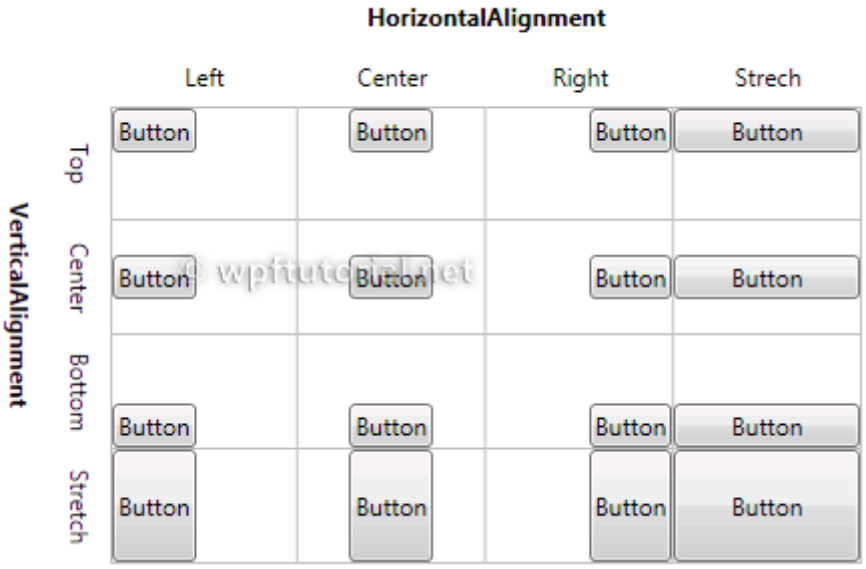
Un layout manager no es más que un delegado que se encarga de organizar los componentes que forman parte de un contenedor como por ejemplo pueda ser una ventana. El layout manager es el encargado de decidir en qué posiciones se renderizarán los componentes, que tamaño tendrán, que porción del contenedor abarcarán, etc. Todo esto se realiza de una manera transparente al programador que por lo tanto se ahorra el tener que escribir una gran cantidad de líneas de control.

Los distintos layouts pueden mezclarse dentro de una misma aplicación para conseguir el efecto deseado, no son excluyentes.

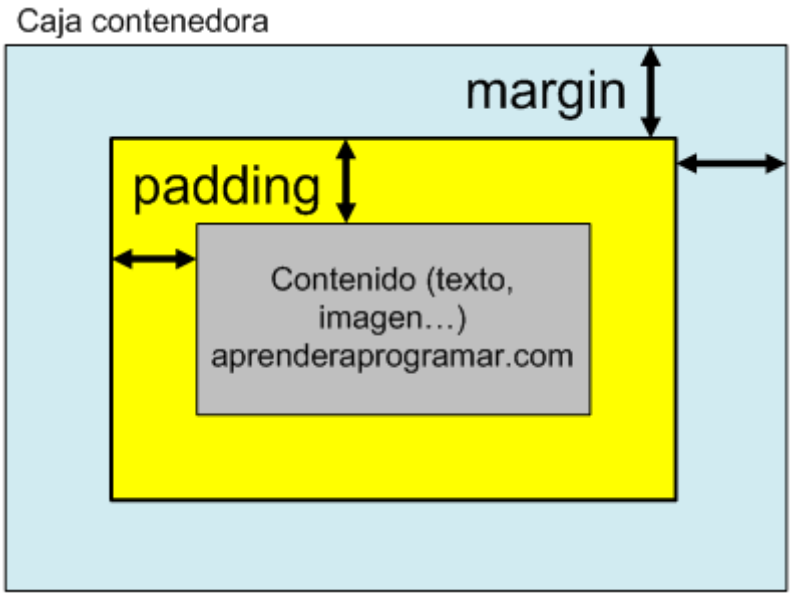
Para que nuestra aplicación resulte versátil y se pueda adaptar a distintas resoluciones, debemos seguir una serie de recomendaciones:

- **Evita las posiciones fijas y utiliza las propiedades de alineación (Aligment) y de margen (Margin) para posicionar los elementos en el panel.**
- **Evita los tamaños fijos para los controles. Utiliza el valor Auto siempre que sea posible.**
- **No abusar de Canvas Panel para situar controles**
- **Utiliza un GridPanel para colocar los componentes de los formularios**

A continuación se muestra una imagen que nos especifica los valores del alineamiento vertical y horizontal:



En la siguiente figura se explica el sentido de las propiedades Margin y Padding:



5.1. GridPanel

Este layout estructura los componentes como una tabla, es decir por filas y columnas. Tiene la capacidad de poder incluir varios componentes en una celda, se pueden expandir entre varias celdas, tanto vertical como horizontalmente.

Por defecto se crea **con una sola fila y columna**, el resto deben de definirse mediante **RowDefinition** o con **ColumnDefinition**.

Para especificar el **tamaño de la fila o columna** podemos especificar un valor **absoluto**, un **porcentaje** o dejarlo con un valor **automático**.

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

Si ponemos la etiqueta **Auto** la columna o fila ocupará el espacio exacto que necesita su contenido. Mientras que si ponemos * entonces la fila o columna ocupará el máximo espacio que pueda.

Si por el contrario queremos que nuestra columna o fila tengan un valor exacto, entonces pondremos el valor numérico, expresado en pixeles, que queremos que tenga.

La última posibilidad nos permite definir porcentajes, es decir, que independientemente del tamaño de nuestra ventana, la aplicación guardará la misma proporción para sus componentes. Por ejemplo, si queremos que una parte de nuestra pantalla ocupe un 30% y la otra un 70%, entonces deberemos utilizar las siguientes expresiones: **3*** y **7***.

Veamos un ejemplo:

```
<Grid>
    <Grid.RowDefinitions>
        <RowDefinition Height="Auto" /> // Automático
        <RowDefinition Height="Auto" />
        <RowDefinition Height="*" />
        <RowDefinition Height="28" /> // Fijo
    </Grid.RowDefinitions>
    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="Auto" />
        <ColumnDefinition Width="200" />
    </Grid.ColumnDefinitions>
</Grid>
```

Para colocar elementos dentro del Grid lo haremos especificando la fila y la columna donde ubicaremos el control:

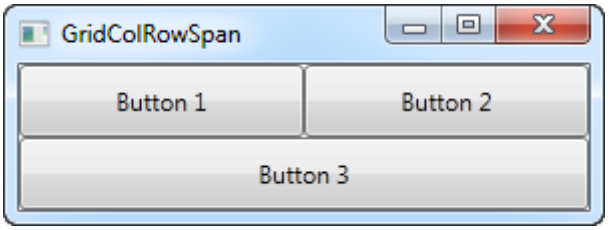
```
<Label Grid.Row="0" Grid.Column="0" Content="Name:"/>
    <Label Grid.Row="1" Grid.Column="0" Content="E-Mail:"/>
    <Label Grid.Row="2" Grid.Column="0" Content="Comment:"/>
    <TextBox Grid.Column="1" Grid.Row="0" Margin="3" />
    <TextBox Grid.Column="1" Grid.Row="1" Margin="3" />
    <TextBox Grid.Column="1" Grid.Row="2" Margin="3" />
    <Button Grid.Column="1" Grid.Row="3" HorizontalAlignment="Right"
            MinWidth="80" Margin="3" Content="Send"  />
```

También dispone de la posibilidad de expandir un control entre varias filas y/o columnas. Veamos un ejemplo:

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

```
<Window x:Class="WpfTutorialSamples.Panels.GridColRowSpan"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="GridColRowSpan" Height="110" Width="300">
        <Grid>
                <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="1*" />
                        <ColumnDefinition Width="1*" />
                </Grid.ColumnDefinitions>
                <Grid.RowDefinitions>
                        <RowDefinition Height="*" />
                        <RowDefinition Height="*" />
                </Grid.RowDefinitions>
                <Button>Button 1</Button>
                <Button Grid.Column="1">Button 2</Button>
                <Button Grid.Row="1" Grid.ColumnSpan="2">Button 3</Button>
        </Grid>
</Window>
```

El resultado es el siguiente:



Como podemos observar en el recuadro rojo, colocamos una propiedad que indica que el componente ocupará 2 columnas, podríamos hacer que ocupara 3, 4, …

5.2. StackPanel

Se trata de un layout sencillo que se encarga de realizar una lista de componentes. La forma de colocación de los controles puede ser en vertical, por defecto, o en horizontal.

Suele ser útil para colocar un conjunto de controles de forma ordenada y sencilla como por ejemplo una serie de botones.

Veamos dos ejemplos:

```
<StackPanel>
  <TextBlock Margin="10" FontSize="20">How do you like your
coffee?</TextBlock>
  <Button Margin="10">Black</Button>
  <Button Margin="10">With milk</Button>
  <Button Margin="10">Latte machiato</Button>
  <Button Margin="10">Chappuchino</Button>
</StackPanel>
```

Ahora veremos la forma horizontal:

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

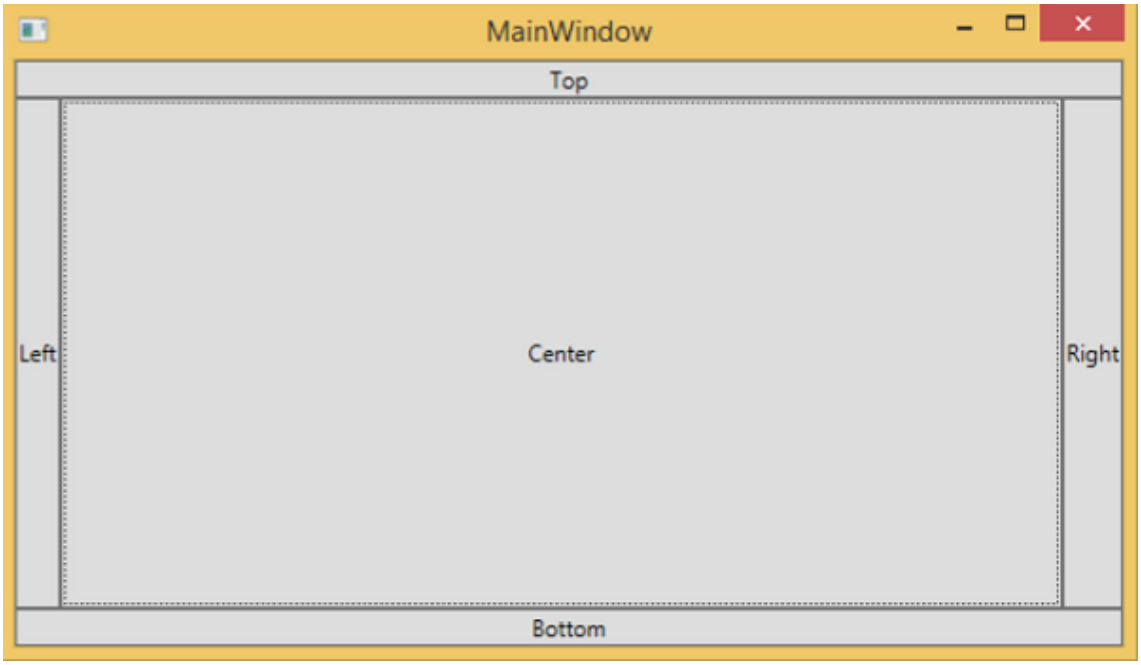
```
<StackPanel Margin="8" Orientation="Horizontal">
   <Button MinWidth="93">OK</Button>
   <Button MinWidth="93" Margin="10,0,0,0">Cancel</Button>
</StackPanel>
```

Como se observa, se trata de un layout que apenas tiene configuración y los componentes se colocan de manera fácil. Sin embargo, no se debe de abusar de este tipo de layout porque tienes algunas carencias, como por ejemplo que no se puede colocar una barra de Scroll dentro de él.

5.3. DockPanel

Se trata de un layout que nos permite dividir la ventana en regiones o zonas donde colocar los elementos.

Divide la pantalla en cinco regiones: *top, bottom, left, right* y *center*. En la siguiente figura se observa las distintas regiones en la pantalla:



El código:

```
<DockPanel LastChildFill="True">
    <Button Content="Dock=Top" DockPanel.Dock="Top"/>
    <Button Content="Dock=Bottom" DockPanel.Dock="Bottom"/>
    <Button Content="Dock=Left"/>
    <Button Content="Dock=Right" DockPanel.Dock="Right"/>
    <Button Content="LastChildFill=True"/>
</DockPanel>
```

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

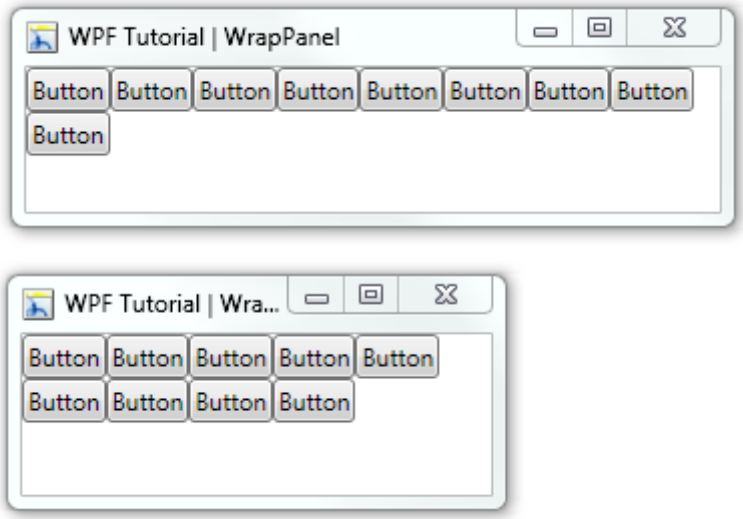
Se utiliza normalmente para **organizar la ventana principal** de una aplicación. De esta manera podemos distribuir los componentes adecuados por cada una de las regiones. **Dentro de este layout podemos utilizar otros tantos**.

5.4. WrapPanel

Es muy similar a un Stack Panel, es decir, amontona los componentes según se van añadiendo al panel.

La diferencia radica en que este layout añade los elementos en la misma línea hasta que no hay sitio y salta a la siguiente.

También podemos elegir la orientación entre horizontal y vertical.



Suele utilizarse para configurar pestañas en un **TabControl,** añadir elementos a una barra de herramientas, …

5.5. CanvasPanel

Es el más básico de los layouts ya que los controles se posicionan en función de las coordenadas que establezca el usuario. Las coordenadas se especifican relativas a cualquiera de los lados del panel utilizando *Canvas.Left, Canvas.Top, Canvas.Bottom, Canvas.Right*.

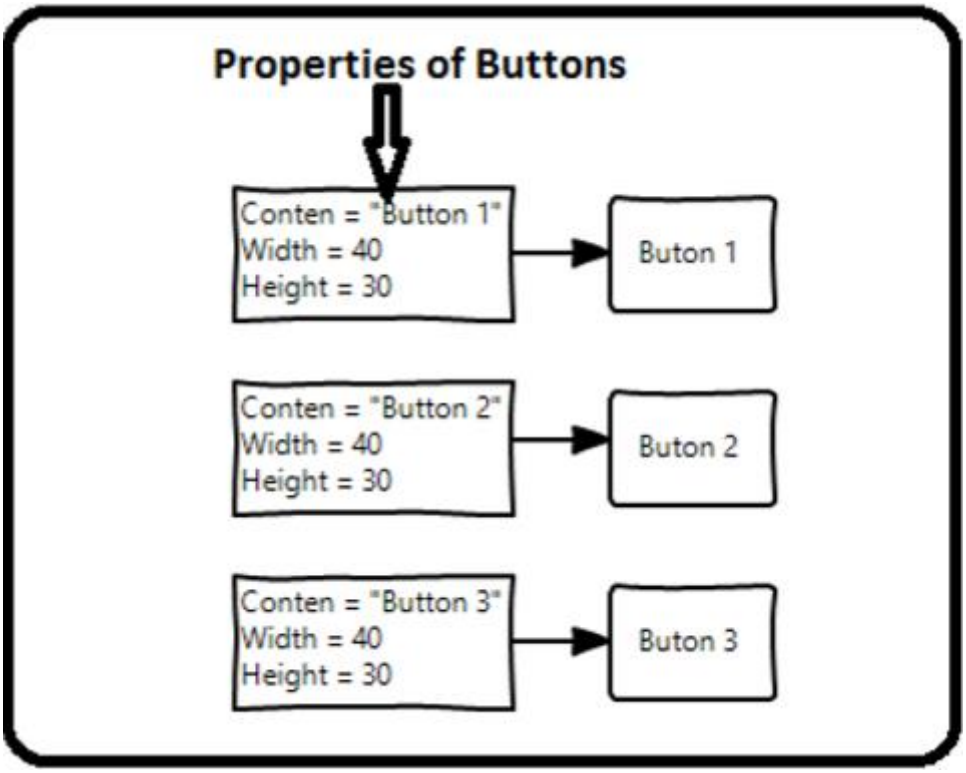
Suele **utilizarse para representar los componentes 2D relacionados con el dibujo de formas**, ya que no es recomendable utilizarlo para colocar controles en la interfaz porque al utilizar posiciones absolutas puede haber problemas cuando se redimensione la ventana.

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

```
<Canvas>
    <Rectangle Canvas.Left="40" Canvas.Top="31" Width="63"
Height="41" Fill="Blue"  />
    <Ellipse Canvas.Left="130" Canvas.Top="79" Width="58" Height="58"
Fill="Blue"  />
    <Path Canvas.Left="61" Canvas.Top="28" Width="133" Height="98"
Fill="Blue"
          Stretch="Fill" Data="M61,125 L193,28"/>
</Canvas>
```

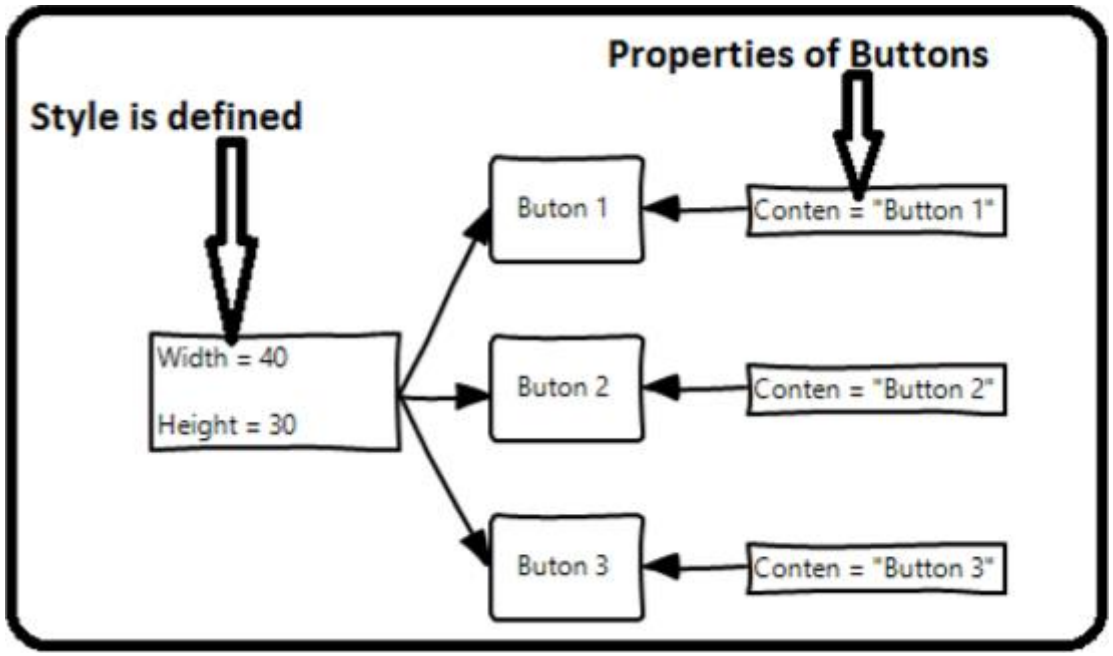
6. Estilos

Como hemos visto en los ejemplos anteriores para poder personalizar los distintos componentes de una aplicación debemos añadir las propiedades en cada uno de los controles, aunque sean del mismo tipo.



Para evitar que tengamos que replicar muchas propiedades podemos utilizar los **estilos** en WPF. Definimos un estilo y lo podemos aplicar a los controles que queramos en toda la aplicación, en una pantalla o en un panel.

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es



En el estilo podemos definir propiedades como el color de fondo, la anchura, la altura, la alineación, etc.

6.1. Niveles

Los estilos se definen, como los controles en la ventana, mediante el lenguaje XAML. Además tiene varios ámbitos de aplicación:

- **Control**: el estilo definido en este nivel solamente se aplicará al control al que afecta.

```
<TextBlock Text="Style test">
    <TextBlock.Style>
        <Style>
            <Setter Property="TextBlock.FontSize" Value="36" />
        </Style>
    </TextBlock.Style>
</TextBlock>
```

Se utiliza la propiedad **Style** del control. A diferencia de los niveles siguientes que se utiliza la propiedad **Resources**.

- **Layout**: en este caso, se aplicará a todo el panel en el que se define el estilo.

```
    <StackPanel Margin="10">
        <StackPanel.Resources>
            <Style TargetType="TextBlock">
                <Setter Property="Foreground" Value="Gray" />
                <Setter Property="FontSize" Value="24" />
            </Style>
        </StackPanel.Resources>
        <TextBlock>Header 1</TextBlock>
        <TextBlock>Header 2</TextBlock>
        <TextBlock Foreground="Blue">Header 3</TextBlock>
    </StackPanel>
```

A todos los TextBlock del Layout se le aplicará el estilo

- **Window**: de la misma forma se aplicará a todos los controles afectados dentro de la ventana.

```
<Window.Resources>
        <Style TargetType="TextBlock">
            <Setter Property="Foreground" Value="Gray" />
            <Setter Property="FontSize" Value="24" />
        </Style>
</Window.Resources>
```

- **Application**: en este nivel afectará a toda la aplicación.

```
    <Application.Resources>
        <Style TargetType="TextBlock">
            <Setter Property="Foreground" Value="Gray" />
            <Setter Property="FontSize" Value="24" />
        </Style>
    </Application.Resources>
```

Para que el estilo se aplique a toda la aplicación debemos modificar el fichero **App.xaml.**

También cabe la posibilidad de establecer los estilos en un fichero aparte para no tener que repetirlos y ubicarlos en un punto centralizado de la aplicación.

Para ello debemos crear un fichero ***Resource Dictionary*** que creamos y dentro de él añadimos los estilos que queramos.

Luego lo llamamos desde la aplicación o desde la ventana con la siguiente sintaxis:

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

```
<Application.Resources>
  <ResourceDictionary>
    <ResourceDictionary.MergedDictionaries>
      <ResourceDictionary Source="Recursos/Estilos/Estilos.xaml"/>
      </ResourceDictionary.MergedDictionaries>
    </ResourceDictionary>
</Application.Resources>
```

6.2. Destino

Una vez establecido el nivel de aplicación del estilo debemos determinar sobre que controles se aplica.

Para especificar el tipo de control sobre el que se modificará el aspecto debemos utilizar la etiqueta **TargetType**. ***Por defecto se aplicarán a todos los controles del nivel especificado***.

*¿Qué sucede si queremos aplicar el estilo solamente a algunos de los controles del tipo especificado por la propiedad **TargetType** que hay en la aplicación?* Por defecto se aplicarán a **todos** los controles.

Para poder flexibilizar la aplicación del estilo dentro de un mismo nivel utilizaremos la propiedad **x:Key**. A continuación vemos un ejemplo:

```
<Window x:Class="WpfTutorialSamples.Styles.ExplicitStyleSample"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presen
tation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="ExplicitStyleSample" Height="150" Width="300">
    <Window.Resources>
        <Style x:Key="HeaderStyle" TargetType="TextBlock">
            <Setter Property="Foreground" Value="Gray" />
            <Setter Property="FontSize" Value="24" />
        </Style>
    </Window.Resources>
    <StackPanel Margin="10">
        <TextBlock>Header 1</TextBlock>
        <TextBlock Style="{StaticResource HeaderStyle}">Header
2</TextBlock>
        <TextBlock>Header 3</TextBlock>
    </StackPanel>
</Window>
```

En este ejemplo vemos que el estilo se aplicará sobre todos los controles del tipo **TextBlock** que tengan definido en la etiqueta **Style** el identificador *HeaderStyle*, solamente a esos se les aplicará el estilo.

Se utiliza la etiqueta **Setter** para asignar las propiedades.

Acceso a los estilos desde C#

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

Podemos definir estilos en nuestro fichero xaml y luego necesitar aplicarlo desde nuestro manejador de eventos. Mediante el método **FindResource**.

```
Style style = Application.Current.FindResource("LabelTemplate") as
Style;
label1.Style = style;
```

Es importante comprobar si el estilo es nulo antes de aplicarlo.

6.3. Triggers

Los estilos pueden activarse o desactivarse en función del valor de una propiedad del objeto. Por ejemplo, al pasar el puntero del ratón por un control podemos cambiar el estilo, y después de que el puntero haya dejado el componente volver al estilo original. Para realizar estas acciones con los estilos utilizamos esta funcionalidad llamada **Triggers**. Existen tres tipos.

6.3.1. Property Trigger

Es el disparador más común y suele utilizarse para determinar un estilo en función del valor de una propiedad.

```
<TextBlock.Style>
    <Style TargetType="TextBlock">
        <Setter Property="Foreground" Value="Blue"></Setter>
            <Style.Triggers>
                <Trigger Property="IsMouseOver" Value="True">
                    <Setter Property="Foreground" Value="Red" />
                    <Setter Property="TextDecorations"
Value="Underline" />
                </Trigger>
            </Style.Triggers>
    </Style>
</TextBlock.Style>
```

6.3.2. Data Trigger

Este desencadenador suele utilizarse para asociar la aplicación del estilo con propiedades de otros componentes.

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

```
<TextBlock.Style>
  <Style TargetType="TextBlock">
    <Setter Property="Text" Value="No" />
      <Setter Property="Foreground" Value="Red" />
        <Style.Triggers>
          <DataTrigger Binding="{Binding ElementName=cbSample,
Path=IsChecked}" Value="True">
            <Setter Property="Text" Value="Yes!" />
            <Setter Property="Foreground" Value="Green" />
          </DataTrigger>
        </Style.Triggers>
  </Style>
</TextBlock.Style>
```

En este caso se enlaza el Trigger con la propiedad **checked** del control denominado **cbSample**, que en este caso es un **CheckBox**.

6.3.3. Event Trigger

En este caso se activa el estilo al activarse un evento y no al alcanzar una propiedad un valor. Suelen utilizarse para crear animaciones o efectos especiales.

```
<Style TargetType="TextBox">
  <Style.Triggers>
    <EventTrigger RoutedEvent="GotFocus">
      <BeginStoryboard>
        <Storyboard>
          <ColorAnimation Storyboard.TargetProperty="Background.Color"
                    To="Gold" Duration="0:0:0.25"/>
        </Storyboard>
      </BeginStoryboard>
    </EventTrigger>

    <EventTrigger RoutedEvent="LostFocus">
      <BeginStoryboard>
        <Storyboard>
          <ColorAnimation Storyboard.TargetProperty="Background.Color"
                     To="AntiqueWhite" Duration="0:0:0.25"/>
        </Storyboard>
      </BeginStoryboard>
    </EventTrigger>
  </Style.Triggers>
</Style>
```

6.3.4. MultiTrigger

Si tenemos la necesidad de especificar más propiedades por las que se activará el estilo utilizaremos la posibilidad de establecer condiciones sobre distintas propiedades.

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

```
<Style TargetType="TextBox">
  <Style.Triggers>
    <MultiTrigger>
      <MultiTrigger.Conditions>
        <Condition Property="IsMouseOver" Value="True"/>
        <Condition Property="Text" Value=""/>
      </MultiTrigger.Conditions>
    <MultiTrigger.Setters>
      <Setter Property="Background" Value="Orange"/>
      <Setter Property="FontSize" Value="15"/>
    </MultiTrigger.Setters>
  </MultiTrigger>
 </Style.Triggers>
</Style>
```

Primero definimos las condiciones que deben de cumplirse y luego establecemos las propiedades que deben de cambiarse.

No se pueden tener varias condiciones con **EventTriggers**, solamente con **Property Triggers** y con **Data Triggers**.

7. Bibliografia

https://msdn.microsoft.com

http://www.wpftutorial.net/

http://www.wpf-tutorial.com/